

Overview

Your Team & contact information

- Célien Grozel celien.grozel@rwth-aachen.de
- Fachrial Perdana fachrial.perdana@rwth-aachen.de

Your Slack Channel (to ask any questions & schedule calls)

<https://holo-rwth.slack.com/archives/C0BDVGUFWGH>

Final presentation link:

...

Your Git repository:

<https://github.com/FDims/location-based-webxr>

Create a fork of <https://github.com/cs-util-com/location-based-webxr> where we will work on (and hopefully also create a few PRs back in the source repo). Your feature ultimately lives **inside the recorder app** (`GpsPlusSlamJs_RecorderApp`), but — exactly like the *Plan First, Then Prototype* workflow below — you first build it as a small **standalone example/prototype** (the quickest way to reuse and extend the **Ray-Triangulation app you already built solo in Phase 1**), prove it there, and only then integrate it into the recorder and open a PR.

Development Process Details

- This Drive document should be the central place where all questions and ToDos are collected so that later any new person could read and understand how this component was developed and what it is about. In a real environment a task/ticket system like Jira would be used but for the lab we will use Google Drive for simplicity.
- You can leave [notes, questions and tasks in this document and assign them to someone](#) so that current responsibilities are always clear.
- Apply Pair Programming in your weekly coding sessions.
 - These sessions are not optional and most of your weekly hours spent for the lab should be in these pair programming sessions.
 - Pair programming sessions can be done anywhere (you don't need to do them in our office) but should happen in person.
 - A single pair programming session for 4 hours is normally much more effective than 2 sessions for 2 hours, keep this in mind while discussing when you want to meet.
 - Use the **Driver / Navigator** model: one person types (the **Driver**, focused on writing code and running tests), the other reviews in real time (the **Navigator**, focused on the big picture — thinking ahead, catching mistakes, looking up documentation). Swap roles regularly (every ~30–60 minutes or after each finished logical task) so both of you spend meaningful time in both roles. The Navigator is not passive — actively question, discuss and guide. If one of you has been silent for 10 minutes, something is wrong.
 - Pair Programming Instructions, Tips & Guidelines: [Copy of The HoloBuilder Development System](#)
- For some tasks we will do code-reviews (good read why we do this: <https://goo.gl/Xvb2Xy>) based on your commits, that's why it is important to always do code refactorings whenever something new is done, write meaningful commit messages and commit regularly.

- **Don't** commit after x hours of work (or even worse at the end of your day) but instead whenever a logical section in the code is finished or a next step on your Todo list is done.
 - This became even more relevant with the use of LLMs to split their work into digestible (and revertable) chunks. Your git history is your undo redo history and commits should be atomic modifications to the state of your code that you can individually review and revert to. Every time you made relevant progress and your test harness is green again you should commit/save your changes into git
- Similar to logical components, commits should contain only one task so if you changed multiple unrelated things in your code commit them separately
- Don't mix refactorings and logical changes of your code. So **first** make a commit where you added something new and then a **second** commit where you cleaned up your code, did some restructuring and added additional comments and documentation where needed.
- Your regular commits and changes to the drive documents will document your weekly progress. Please keep this document up to date with all your progress, ideas etc.
- When contributing back to the upstream open source project, work in your fork on a feature branch and follow the project's existing code style and conventions (even where they differ from your own preferences). Run the project's tests before opening a PR and write a clear PR description that references the related issue. Maintainer feedback and requested changes are normal and expected — respond to them. Always read through your created PR (the proposed changes) to see if there are any changes that should be reverted/cleaned up to minimize the changes where possible. Get one small PR through the full process first before tackling bigger changes.
- We meet for regular review calls (roughly every two weeks). Come prepared: what you worked on since the last call, what challenges you ran into, what you learned, and what you plan to do next. Both team members should be active in these calls — it is not a one-person update.

Working with a Product Owner (important context)

For this project Simon will act as your **Product Owner**, a person who sits between you and the "customer". He brings **high-level goals, customer feedback, and ideas worth investigating**, not detailed technical instructions; turning those into a concrete, well-architected solution is your job. The point is to simulate what working in a real product team feels like, where requirements are rarely complete up front and keep evolving as you and the customer learn more. How well you operate in this setup — not just the code you produce — is part of what is being evaluated.

What this means for how you work:

- **Requirements will be high-level and sometimes deliberately vague or incomplete.** When something is unclear, that is usually not an oversight to silently work around — it is a prompt to **ask**. Collect open questions (as you already do in Task 1) and bring them to us. Coming with a couple of concrete options and their trade-offs ("we could do A, simpler, or B, more flexible but slower — which matters more to the customer?") is far more useful than guessing or asking a fully open-ended question.
- **The spec is a living backlog, not a fixed contract.** The vision in §2.1 and the component list in §2.3 are today's best understanding. Expect priorities to shift and features to be added, cut, reordered, or revisited as customer feedback comes in. This is normal and expected — reacting well to change is exactly what is being practiced here, not a sign that something went wrong.
- **Demo early and often to get feedback cheaply.** The "build each component on its own and show it to us" loop (Goal 1 in Task 2) *is* the feedback mechanism: each demo is a chance to see if the customer reacts positively and adjust direction if needed *before* you have over-invested. A rough early demo that surfaces a misunderstanding is worth more than weeks of polished work in the wrong direction. This is also *why* you build small, independently replaceable components — it keeps the cost of a changed requirement contained to one component instead of forcing a rewrite.
- **Push back, and bring your own ideas.** A good product team is a two-way conversation. If you think a requested feature is a bad idea, technically expensive, or that there is a better approach, say so with reasoning — challenge the product direction the same way you learned to challenge an LLM's plan. Likewise, if you discover something interesting or a hard limitation while building (for example during the Task 1 recordings), raise it; it may change what is worth building.

- **Be explicit about scope and trade-offs.** When new feedback arrives, name its cost ("if we add this, this other component slips by about a week") so the product decision can be made together with eyes open. Use the Time Estimations table and the review calls for exactly this.

The regular review calls and your component demos are the main venues for this feedback loop, so treat them as customer reviews: come ready to show something working, ask your open questions, and agree on what to build next.

Development Workflow: Plan First, Then Prototype

A core idea in software engineering is to build software out of **small, independent components** with **clear interfaces** between them. Each component is developed, tested, and maintained on its own, and only integrated with the others once it is complete and proven to do what it should. This keeps complexity manageable, lets your team work on different components in parallel, and — crucially — means any single component can later be **replaced with an improved version** without touching the rest of the app, as long as its interface stays the same.

The *Plan First, Then Prototype* workflow fits naturally into this approach: it lets you nail down and validate one component at a time in isolation. It also makes AI-generated code far less risky — if an LLM produces a component that works but is a bit too verbose or just not maintainable over time, that stays an **isolated, replaceable problem of that one component** (you can redo or swap out just that piece of the entire system) rather than something that eventually forces a rewrite of the whole application.

Before building any new component, follow this workflow (it builds on the plan-first approach you already practiced in Phase 2). Don't jump straight into writing the production code in your repo.

1. **Write a detailed plan first.** For each component, create a plan markdown file in the repo (e.g. `2026-12-24-PLAN.md` or `2026-12-24-<component>-plan.md`) that captures, *before* any production code:
 - what the use case is about and the problem it solves,
 - what you want to achieve — the concrete goals, requirements, and how you will know it works (success criteria),
 - the intended approach / architecture and the key open questions.
 - It is fine — and encouraged — to include example code snippets in the plan to make an idea concrete.
2. **Iterate on the plan with an LLM.** Feed the plan to an LLM and ask it to act as a critical reviewer: poke holes, surface edge cases, and propose alternatives (the same "make the AI challenge you" skill from Phase 2). Do a few rounds and refine the plan until no major problems are left. Commit each meaningful revision so the plan's evolution is visible in git.
3. **Generate several throwaway prototypes.** Once the plan is solid, hand the *same* plan to **multiple different LLMs** and have each one build a small, self-contained, **testable prototype** — just enough to run and try, not the final system. You can do this either in VS code, in the Gemini or OpenAI web uis or any other tool you prefer where web examples can efficiently be prototyped and tested.
4. **Try them and pick the best.** Run the prototypes and compare them on **both axes**: the UX (how it looks and feels to use) and the code (is it clean, understandable, and reasonable to build on?). From this review you will typically be able to pick one you prefer most as a starting point.
5. **Build the real thing.** Treat the winning prototype as reference/example code — do **not** just drop it into the repo as-is. Implement a proper, reusable, well-tested version in your actual repository, following the architecture and separation-of-concerns expectations you already know.
6. **Test it at two levels.** Cover the component's pure functions and logic with **unit tests** — they are fast and pin down the logic, math and edge cases. On top of that, add **end-to-end tests** ("e2e" tests) that replay the realistic outdoor recordings you captured in Task 1, so you can verify the whole component behaves correctly in a real outdoor context — deterministically, on a desktop, without manual input by you every time you do a small change. Unit tests get you far, but the replay-driven e2e tests are what prove the system actually works in the field and that everything is plugged in together correctly.

Time Estimations

This section is for you to estimate when you think you will be done with each of the iterations we will do for the component. The first iteration should be as small as possible to get the first prototype running with the minimal amount of needed features. Then we will decide together how to improve this first version. The process in summary:

1. Estimate for yourself how long you think the individual iteration with all its tasks for this component will take.
2. Discuss the estimated days with your team partner and write down a date when you think the first task will be finished.
3. This way you can later check if your initial guesses were roughly correct or if you estimated too much or too little time.
4. After the subtask is done, write down the actual date next to the estimated date. (If you forget this step use the git history to later fill this in)
5. Whenever an iteration is done we do the same process again, discuss your estimated dates again and write down the discussed date for the new iteration.
6. This way you will learn to estimate these iterations more accurately, and since most projects/initiatives have deadlines being able to (gu)estimate tasks to a rough realistic completion date and with that being able to decide if a certain feature can be included into the project or needs to be left for a future iteration will be a valued and regularly needed skill.

Please put in your estimated timeframe below and for yourself as a learning to get better with estimating the needed time in each iteration also track how long you actually needed:

Iteration Description (Short list which features will be in the planned version)	Estimated completion date	Actual completion date

Final Presentation

At the end of the lab your team gives a short presentation (~15–20 minutes including Q&A) to the whole group. Both of you present and answer questions, so keep notes throughout the lab to make preparing it easy. Cover:

- **The component:** what it does and why it is useful for the location-based-webxr project.
- **Your contributions:** what you built/changed, ideally with a live demo or a short code walkthrough.
- **Architecture:** how your component is structured and how it fits into the library.
- **Lessons learned (focus here):** what was hard, what surprised you, what you would do differently, and how pair programming worked for you.
- **AI reflection:** how you used AI tools, and how contributing to an existing codebase changed that compared to building something from scratch.

Tasks

Task 1 - Getting to know the AR framework

1. <https://github.com/cs-util-com/location-based-webxr> will fit well for the purpose of the lab since its still in its early development phase and fits both our lab focus and tech stack quite well.

1. We will start with a few smaller modular components that can be developed and tested in any [three.js](#) project but make sense especially with Augmented Reality applications in mind
2. To learn the details of the [location-based-webxr](#) library first try out its demos and do a few recordings via <https://gps.csutil.com/recorder/> in an outdoor environment of your choice. This will also generate realistic outdoor usage data that can be used when testing your developed components, but in general the goal of this step is that you make yourself familiar with what the library is about and what its limitations are (e.g. indoor only will not work).
 1. **Record 5 walks** of about 5-10 minutes and mark the same reference points in these every time. The reference points you place in the first walk should have at least 50m distance to each other and should be put on spots that you can easily mark again in the walks you do afterwards. Mark at least 3-5 reference points on your first walk.
 2. In the second recording, walk the same path back and mark all reference points again. Standing as exact as possible in the same spot/phone position as you did in the first walk is very important since it serves as the ground truth for validating how accurate this gps+webxr fusion system works.
 3. Observe how accurate all reference points are displayed (they are rendered as 3D markers anchored at the location). Document your observations, how often are the displayed reference point markers (spheres flying where the reference point was captured) in the correct spot, how often are they only close by, how often are they entirely off. Understanding this is essential to understand what use cases are possible with this system and what might not work well enough.
 4. If you have two capable Android phones feel free to do the same workflow on both devices in parallel
3. The instructions above are still a bit vague on what the apps are about and what they are used for — collect any questions you have while doing these tasks, both high level and detailed ones.
 1. Put these collected notes into a developer experience feedback md file in the repository and keep the points even after you have found answers to them.
4. There is a part of Task 1 that is **specific to your team**, because your feature both *extends* the recorder's existing reference-point flow and *reuses* its depth sampling. While recording, pay deliberate attention to three things and write down what you notice:
 1. **How a reference point is marked today.** The current "Mark Ref Point" flow records the point at the **device's own pose** — i.e. you physically put the phone *on* the landmark (a bench corner, a statue base). Notice the cases where that is awkward or impossible: a point **up high, across a street, or 100+ m away** that you simply cannot walk the phone onto. That gap — marking points you *cannot physically touch* — is exactly what your Measurement Points feature is for.
 2. **How depth behaves.** The recorder samples a depth map / occupancy grid. Aim at things near and far and get an intuition for **where depth is trustworthy and where it falls apart** (close surfaces vs. far-away or thin/textured targets, bright sky, etc.). Your feature leans on depth up close and must *stop* trusting it at range — feeling this first-hand will make the weighting in §2.5.3 concrete.
 3. **The link back to your Phase 1 app.** Re-read your own **Ray Triangulation 3D** project from Phase 1. The core you wrote there — accumulate rays, compute their closest point of approach, show an error/accuracy metric — is the seed of this whole task (see the callout in Task 2). Note what you would keep and what you would change now that the rays come from a real moving phone instead of a virtual fly-camera.

Task 2 - Adding a Measurement Points feature to the recorder app (triangulation + depth fusion)

You are adding a **Measurement Points** feature to the recorder app (`GpsPlusSlamJs_RecorderApp`): a way for the user to mark a point in **3D space that they cannot physically touch** — a spot up high, across the street, or hundreds of meters away — by **shooting rays at it from different positions** and letting the app triangulate where those rays best meet, **fused with the phone's depth sensing**. It is the natural next step beyond the existing reference-point flow, which can only mark places you can walk the phone onto.

This is the Phase 1 Ray-Triangulation app, grown up. In Phase 1 you each built — solo, without AI — a tool where you fly a virtual camera, **shoot rays** (`addRay` captured the camera pose), **accumulate** them, **compute their closest point of approach** (a least-squares / shortest-distance-between-lines problem), spawn a **sphere** at the result, and show a

self-designed **error/accuracy metric** of how well the rays converged — all with a clean split between pure math, the Redux store, and the views. **Almost all of that transfers directly.** The differences are: the rays now come from the **real device pose** as you physically move (so *walking sideways* is what gives you the triangulation baseline, instead of orbiting a fly-camera); the estimate is **fused with the depth map**; outliers are rejected with **RANSAC**; and the result is **persisted and visualized in the recorder's world**. So the genuinely *new* research is narrow and well-bounded — depth fusion + robustness — sitting on top of an estimator you have already shipped once.

There are **two use cases** to keep in mind, and a good solution handles both with one mechanism:

- **Short range (depth-dominated).** The target is a few meters away, the depth map is reliable, and even a single well-aimed ray plus its depth reading already pins the point down well. More rays refine it.
- **Long range (parallax-dominated).** The target is **100–200 m** away, where depth is useless. The user marks it, **walks several meters to the side** (say ~10 m), and marks it again — and again. With a wide enough **baseline** between those vantage points, the rays triangulate the far point even though no single measurement could.

Because it is a real feature with several moving parts, the point of this task is to **practice the "small, independent components" idea for real**: build the pure estimator, the depth provider, the AR interaction and the persistence each as an isolated, individually testable piece with its own tiny demo, prove the whole thing first as a **standalone example/prototype**, and only then **integrate it into the recorder** and PR it back.

Treat the destination in §2.1 as the goal, the contract in §2.2 as what lets you split the work, and the component list in §2.3 as the menu of pieces — **the order you build them in is yours to decide and defend** (see §2.3 and §2.6).

2.1 The product in one paragraph (the destination)

While recording, the user points the phone at something they want to mark — a fixed **crosshair** in the middle of the screen, or a **tap** on the specific target in the camera feed — and "shoots" a ray. The app captures the **real AR device pose** at that instant (the same pose stream the recorder already reads every frame) and turns it into a ray into the world, drawn as a line, and at the same moment **samples the depth map along that exact aimed direction** to get a rough distance to the target. From the **very first** ray it already shows a **provisional measurement sphere** — seeded by that depth reading — with a live **uncertainty / convergence** readout next to it. As the user **moves and shoots more rays** from different positions, the app **re-solves** the point as the closest point of approach of all the rays, with the depth readings folded in as a **distance-weighted prior** (trusted up close, fading out with range, and progressively **out-voted** by the triangulation as the baseline grows), and with **robust outlier rejection** discarding bad rays. The sphere tightens and the uncertainty shrinks; for far targets the app **actively coaches** the user ("move sideways and mark again") until the baseline is wide enough. When happy, the user **confirms** the point. It is stored as a new **Measurement Point** — reusing the recorder's per-scenario point storage and its GPS-world sphere visualization — and rendered as **two connected dots**: one in the AR-local frame and one in the GPS-world frame, the small gap between them doubling as a live **alignment-quality probe**. Its primary purpose is to be a **handy in-the-field measuring/marketing tool** for points you cannot reach; the cross-session persistence and the alignment diagnostic come along essentially for free.

2.2 The architecture that makes this splittable (agree on and document this first)

Everything flows through a few **shared contracts**, and getting them right *before* you split up the work is what lets the two of you build different pieces that combine correctly. Write them down as TypeScript interfaces in your first plan file and review them with us early.

- **The estimator interface (the pure heart, lifted from Phase 1).** A pure function from a set of **observations** — each ray (*origin, direction*) plus an optional depth-derived point and a weight — to a **result** (*point*, plus an **uncertainty / error metric**), all in **one consistent coordinate frame** (see §2.5.2) and with **no Three.js, no DOM, no WebXR**. This is the same closest-point-of-approach solver you wrote in Phase 1, generalized to carry weights and a depth prior. Everything downstream is tested against this interface, and it is the single most reusable artifact you produce (a candidate upstream PR — it is GPS-free, framework-free math).
- **The MeasurementPoint entity (what gets persisted).** Model it on the recorder's existing *RefPointDefinition / RefPointObservation* so you can **reuse the storage and visualization** rather than inventing new ones. A measurement point records the **rays** that produced it (each with the device pose and the depth observation used),

the solved **AR-local position**, the derived **GPS-world position**, the final **uncertainty**, and the usual ids/timestamps/session links. Decide early whether these live in the same per-scenario `refPoints/` directory (with a type discriminator) or a sibling `measurementPoints/` directory — a small schema decision to bring to a review call (§2.6).

- **The Redux action model (reuse the recorder's library store, don't fork it).** Mirror the Phase 1 shape on top of the recorder's existing `gps-plus-slam-js` store: an `addMeasurementRay` action (capture the pose + depth observation), a deliberate **compute/refine** step, `confirmMeasurementPoint`, `deleteMeasurementPoint`, and **undo/redo** — and, like every other recorder action, persisted to disk so the whole marking flow **replays deterministically** (§2.4). Do **not** create a custom store or record rays outside the library store; that breaks replay (see the recorder README's "Common Mistakes").

Once these are agreed, each component below talks only to these contracts, so you can build and test them independently.

2.3 The components (Goal 1 — build and demo each on its own)

For **every** component, follow the *Plan First, Then Prototype* workflow above, and hold yourself to the same bar:

- a clear **interface** (what goes in, what comes out) expressed against the §2.2 contracts,
- **unit tests** for all the pure logic and math,
- where the component depends on movement, **replay e2e tests** that feed it the real outdoor recordings you captured in Task 1 so it runs deterministically on a desktop with no phone and no manual input,
- a **tiny standalone demo** (its own minimal page or scene) you can show *without* the rest of the app existing.

On ordering — this is yours to decide. Unlike the other teams we are deliberately **not** prescribing the iteration sequence here. Part of your task (and your estimation exercise) is to **propose the order yourself and present it with reasoning** at a review call: what is the smallest first iteration, what each later iteration adds, and why. There are only two soft constraints: the **pure estimator (1)** underpins almost everything, and the **recorder integration + replay validation (7)** mostly composes the others, so it tends to come last. A natural *smallest-possible* first slice we discussed is "**one ray + its depth reading → a measured point**" (the degenerate, single-observation case of the estimator — no multi-ray triangulation, no RANSAC, no persistence yet); but if you can argue that leading with the **ported multi-ray triangulation core** is the better first iteration, make that case. Bring your proposed ladder and we decide together.

1. **Ray-triangulation core (pure — ported from your Phase 1 app).** The closest-point-of-approach solver: given N weighted rays in one frame, compute the best-fit point and a meaningful **uncertainty / convergence metric** (reuse and refine the error metric you designed in Phase 1). Generalize it to accept a **depth-derived point + weight** alongside the rays so the later fusion (component 3) has a single clean entry point. No Three.js, no DOM.
 1. *Test:* unit-test against known geometry — two exactly intersecting rays return the intersection with ~zero error; near-parallel rays return a sensible point with **high** uncertainty; adding a third consistent ray lowers the uncertainty; weights bias the result as expected. (This is essentially your Phase 1 test suite, extended.)
 2. *Demo:* a plain scene (no AR) where you place a few rays by hand and see the computed point + error — the Phase 1 app itself, repurposed as the demo harness.
2. **Depth-prior provider (on-demand depth at the aimed pixel).** At shoot-time, sample the recorder's existing depth map / occupancy grid **along the aimed direction** (the crosshair pixel, or the tapped pixel), unproject it to a 3D point on the ray, and attach a **confidence weight that decays with distance** (§2.5.3). Reuse the recorder's depth-sampling rather than adding a parallel one.
 1. *Test:* unit-test the pure unprojection (a known depth value + pixel + camera intrinsics → the expected 3D point on the ray) and the confidence/weight model (weight high for near depth, decaying to ~0 at long range; "no usable depth" handled cleanly). The live depth read is view-layer.
 2. *Demo:* aim at a near object and a far one and print the depth point + its confidence; show the confidence collapsing at distance.
3. **Depth-weighted fusion + robust estimation (the genuinely new heart).** Combine the rays (component 1) and the per-ray depth points (component 2) into one estimate: depth as a **distance-weighted soft prior** — strong up close, fading with range, and progressively **out-voted** by triangulation as the baseline grows — with **mandated robust outlier rejection** (RANSAC is the reference approach; the exact method, sample model and thresholds are **yours to choose and justify**, §2.5.4). The single-ray-plus-depth case (a candidate first iteration) should fall out of this as the degenerate case.
 1. *Test:* unit-test on synthetic scenes with a **known answer**: a clean multi-ray set converges to it; an **injected outlier ray** is rejected and does not move the result; with one ray the depth prior dominates, and as good

rays accumulate the solution moves off the depth prior toward the triangulated point. Determinism where the method allows (seed any sampling).

2. *Demo*: scrub from 1 → N rays (with a deliberately bad one mixed in) and watch the estimate and uncertainty evolve, the outlier get rejected, and depth's influence fade.
4. **Aiming + ray capture in AR (crosshair + tap)**. Turn a "shoot" into a ray: capture the **device pose** at that instant (reuse the recorder's `getCurrentArPose()`), and build the ray either along the **device-forward** direction (a fixed center **crosshair** — point the whole phone) or through a **tapped pixel** (unproject via the camera intrinsics). Support both, crosshair as the steady default and tap for picking a specific target without re-aiming the phone. Draw each ray as a line in the scene.
 1. *Test*: unit-test the pure ray-construction math, especially **tap unprojection** (a tapped pixel + the WebXR projection → the correct world-space ray) and the crosshair = camera-forward case, against known inputs. The live pose read and the line rendering are view-layer.
 2. *Demo*: in AR (or by replaying a walk on desktop), shoot a few rays both ways and see the lines appear from the correct poses.
5. **Live measurement UX + baseline coaching**. The interaction loop: a **provisional sphere** appears from the first ray+depth and **re-solves and tightens** as rays accumulate (the Phase 1 feedback loop, now real-time), with a live **uncertainty** readout; for far targets, **actively coach** the user to build parallax ("move sideways and mark again") and show the uncertainty shrinking as the baseline grows; an explicit **Confirm** commits the point. Keep the *decision* logic pure and testable; the on-screen widgets are view-layer.
 1. *Test*: unit-test the pure state/decision logic — when to show "needs more baseline" vs. "good enough", how the uncertainty maps to the coaching prompt, the provisional→confirmed transition. Feed it synthetic sequences of observations.
 2. *Demo*: replay a walk that marks a far point from several spots and watch the prompt, the shrinking uncertainty, and the tightening sphere — no phone needed.
6. **Measurement-point entity: persistence + dual-frame visualization**. Persist a confirmed point as the §2.2 `MeasurementPoint`, **reusing** the recorder's per-scenario point storage and sphere-visualization infra. Solve in the **AR-local frame**, then store **both** the AR-local position and the **GPS-world** position (via the alignment matrix), and render them as **two connected dots**: the gap between them is a live **alignment-error probe** (the same spirit as the recorder's existing raw-GPS-vs-fused marker pair). Wire `delete` + **undo/redo**.
 1. *Test*: a **round-trip** test (confirm a point → persist → reload → recover an equivalent entity, with both frames intact); the AR↔GPS conversion is correct against a known alignment matrix; undo/redo restores prior state.
 2. *Demo*: mark and confirm a point, reload, and see it come back as the two connected dots; show the gap reacting as the alignment matrix updates.
7. **Recorder integration + replay-based validation (compose last)**. Wire 1–6 into the recorder's record/replay loop: the new actions persist to disk like every other recorder action, so a recorded session (which **already logs depth**) deterministically **replays the entire marking flow** on desktop. This is what proves the feature is plugged in correctly, not just that a live demo happened to work once.
 1. *Test*: an **e2e replay test** — replay a Task 1 recording that marks a point, and assert the same Measurement Point (position within tolerance, uncertainty) is reproduced deterministically. The unit-level math is already covered by 1–6.
 2. *Demo*: the feature running inside the recorder on a phone outdoors **and** the same session replayed on desktop producing the identical point.

2.4 Composition & the end-to-end proof

Once the pieces are signed off individually, the "composition" here is the **integration into the recorder** (component 7): the Measurement Points feature living in `GpsPlusSlamJs_RecorderApp`, marking points during a real recording, persisting them per scenario, and visualizing them as the dual AR/GPS dots — with the standalone example/prototype retained as the fast demo/regression harness.

Prove the whole thing the same deterministic way you proved each part: an **end-to-end replay test** that plays one of your Task 1 recordings through the integrated feature and asserts the marked point comes out where it should, within its reported uncertainty. The on-device run proves it works in the field; the replay test proves it stays working as you refactor — without a phone or a walk outside. For the **long-range** case specifically, a recording where you marked a far landmark from several vantage points is the fixture that proves the parallax/baseline path actually triangulates it.

2.5 Recommended approaches & the hard parts (these double as search terms)

1. **Reuse your Phase 1 estimator — don't rewrite the math.** The closest-point-of-approach / least-squares "shortest distance between several lines" solver and the convergence error metric are exactly what you already built and tested in Phase 1. Lift them in as a pure, framework-free module and *extend* (weights + a depth prior), rather than starting from scratch. Keep it free of Three.js/DOM/WebXR so it stays unit-testable and PR-able. (Search: "closest point of approach multiple rays least squares", "line-line shortest distance 3D", "weighted least squares point triangulation".)
2. **Solve in the AR-local frame, store in both frames — and let the gap be a feature.** All the rays in one marking are captured in the recorder's **single, consistent AR-local frame**, so triangulate **there** (don't couple the solve to the live GPS alignment quality). Then convert the result to **GPS-world** space via the alignment matrix for persistence/cross-session display, and keep the **AR-local** position too — render both as the **two connected dots** whose separation is a live alignment-error/drift probe. Mind the recorder's scene hierarchy: AR-local content lives under `arWorldGroup`, GPS-world content under `scene` (see the recorder README). Watch coordinate conventions (WebXR/Three.js right-handed, Y-up) when you move points between frames. (Search: "three.js world vs local space getWorldPosition", "apply matrix4 transform point", "AR drift alignment error visualization".)
3. **Depth is a *prior*, not an answer — weight it by distance and let triangulation out-vote it.** The depth map is decent up close and **unreliable far away** (and on thin/untextured/bright targets), so fold each depth reading in as a **soft constraint whose weight decays with the measured distance / confidence**, never as a hard position. With a single ray the prior carries the estimate; as more rays widen the baseline, the triangulation should dominate and the depth weight fade. Decide the decay/confidence model empirically from your Task 1 depth observations. (Search: "depth sensing accuracy vs distance", "soft prior weighted estimation", "sensor fusion confidence weighting", "WebXR depth-sensing API".)
4. **Robust estimation is mandatory; the exact method is yours to justify.** Plain least-squares lets one bad ray wreck the result, so you **must** reject outliers — RANSAC is the reference (e.g. a minimal-sample candidate point from a couple of rays, count rays whose closest-approach distance is within a threshold, refit on the inlier set), but the sampling model, the inlier threshold (in meters) and whether you prefer an IRLS/robust-loss variant are **research decisions you make and defend**, not ones we prescribe. Tune thresholds on real recordings, not in the abstract. (Search: "RANSAC line/point fitting", "robust estimation IRLS Huber loss", "outlier rejection triangulation".)
5. **Aiming: crosshair = device-forward, tap = unproject the pixel.** The crosshair ray is just the device's forward axis (cheap, steady, mirrors the Phase 1 "camera-forward" ray). The tap ray needs the camera **intrinsics / projection matrix** to unproject the tapped pixel into a world-space direction — keep that math pure and tested, and verify the tap stays usable at long range on the actual phone (a tiny aim error becomes a large lateral error far away). (Search: "three.js Raycaster setFromCamera unproject", "screen pixel to world ray camera intrinsics", "WebXR view projection matrix".)
6. **Baseline is everything for far points — and it is teachable.** Triangulation uncertainty for a distant target scales roughly with $\text{distance}^2 / \text{baseline}$, so a 150 m point needs *meters* of sideways movement between shots to pin down. Turn that into the **active coaching** signal: show the uncertainty shrinking as the user moves and prompt them to keep building parallax until it is good enough. This is the Phase 1 error metric grown into UX. (Search: "stereo triangulation baseline depth uncertainty", "parallax angle triangulation accuracy", "epipolar baseline error propagation".)
7. **Reuse the recorder's existing seams, don't reinvent them.** The recorder already gives you almost every plumbing piece: `getCurrentArPose()` for the per-frame pose, depth sampling / the occupancy grid, the `refPoints/` per-scenario JSON storage + loader, the `RefPointVisualizer` sphere pattern, the raw-vs-fused `GpsEventVisualizer` (your model for the dual dots), the alignment-matrix subscription, and the action-log replay engine. Build *on* these (and follow the README's "Common Mistakes": use the library store, record paired data, correct scene hierarchy). Where a seam is *almost* but not quite reusable, cleaning it up is itself a candidate upstream PR. (Search the recorder repo for: `getCurrentArPose`, `ref-point-loader`, `RefPointVisualizer`, `GpsEventVisualizer`, `applyAlignmentMatrix`, `depth-sampler`.)

2.6 Open questions to resolve early (bring these to a review call)

- **The iteration order (your call, with reasoning).** As per §2.3: propose the smallest first iteration and the ladder above it, with justification, and we agree it together. (Current lean to confirm: start with "one ray + depth →

point", then add multi-ray triangulation + the soft-prior fusion, then RANSAC, then long-range baseline coaching, then persistence + dual dots, then replay validation — but make the case yourselves.)

- **The depth confidence model.** How fast should depth's weight decay with distance, and does it need any per-device calibration? Decide from your Task 1 depth observations, not on paper.
- **RANSAC thresholds & "converged enough".** What counts as an inlier (in meters), how aggressive is rejection, and at what uncertainty do you suggest the user can **Confirm**? Tune on real recordings.
- **Crosshair vs. tap, and tap reliability at range.** Which is the default, and is tap-to-aim accurate enough far away on the target Android device? Verify on hardware.
- **Where measurement points live in the scenario folder.** Same `refPoints/` directory with a `type` discriminator, or a sibling `measurementPoints/` directory? A small storage-schema decision that affects reuse of the existing loader/visualizer.
- **Scope of the dual-dot alignment probe.** Is the AR↔GPS gap purely a measurement-feature visualization, or should it be a more general recorder diagnostic? Keep it contained for now unless we decide otherwise (the headline purpose stays a *measuring tool*).

2.7 Future iteration ideas

1. **Measure between points.** Once you can mark untouchable points, the obvious next tool is **distance/area between measurement points** — a real-world tape measure / area tool built on the same entity.
2. **Feed the accuracy-validation story.** Measurement points capture *untouchable* ground-truth that the touch-the-phone reference points cannot — fold them into the same cross-session alignment-accuracy validation the reference points already serve (their convergence across sessions becomes another quality signal).
3. **Better long-range fusion.** For very far targets, bring in **GPS anchors** and **multi-session** triangulation (the same point marked across different recordings) to close baselines a single walk cannot.
4. **Active vantage-point guidance (coaching v2).** Don't just say "move sideways" — compute and show *where* the next shot would most reduce uncertainty (the direction of maximum parallax gain).
5. **Patch-based depth.** Sample a small neighborhood around the aimed pixel and fit a local plane/normal, for a more robust depth prior on flat surfaces and an orientation, not just a point.
6. **Contribute the estimator upstream.** The pure triangulation + depth-weighted-fusion + robust-estimation module is GPS-free, framework-free math useful to any AR project on top of `location-based-webxr` — arguably the most reusable thing you produce, and a strong PR.

Appendix: Quick orientation (your Phase 1 code + the recorder seams)

A few concrete starting points so you don't begin from a blank page (confirm against the current recorder source — it evolves).

Your Phase 1 app is the seed. The Ray-Triangulation 3D project you built solo already contains: ray accumulation, the closest-point-of-approach solver, the convergence/error metric, the `addRay` / compute action model, sphere rendering, delete and undo/redo, and the pure-math / state / view separation. Start by lifting the **pure solver + error metric** out of it into the §2.2 estimator interface — that is component 1, mostly done.

The recorder gives you the plumbing (`GpsPlusSlamJs_RecorderApp/src/...` — see its README):

- `ar/webxr-session.ts` — the per-frame pose stream and `getCurrentArPose()` (your ray origin/direction source), the scene → `arWorldGroup` → camera hierarchy, and `applyAlignmentMatrix()`.
- depth sampling / occupancy grid — the source for your **on-demand depth at the aimed pixel** (component 2).
- `storage/ref-point-loader.ts` and the `RefPointDefinition` / `RefPointObservation` schema — the per-scenario JSON storage you **reuse** for `MeasurementPoint`.

- visualization/reference-points.ts (RefPointVisualizer) — the sphere-in-GPS-world pattern; and visualization/gps-event-markers.ts (GpsEventVisualizer, raw-yellow vs fused-cyan) — your model for the **two connected dots**.
- the library store + action-log replay engine — record the new actions through the **library store** (not a custom one) so the marking flow **replays deterministically** for tests.

A tiny starter for the two-ray closest point (the kernel of component 1 — read it together with §2.5.1, then generalize to N weighted rays and add the depth prior):

```

• import { Vector3 } from 'three';
•
• // Closest point between two rays  $a(s) = a_0 + s \cdot da$  and  $b(t) = b_0 + t \cdot db$ 
• // ( $da, db$  assumed normalized). Returns the midpoint of the shortest segment
• // connecting them, plus that gap distance as a first, crude error signal.
• // This is the 2-ray case; the real solver least-squares-fits N (weighted) rays.
• function closestPointTwoRays(a0: Vector3, da: Vector3, b0: Vector3, db: Vector3) {
•   const r = new Vector3().subVectors(a0, b0);
•   const daDb = da.dot(db);
•   const denom = 1 - daDb * daDb;           // 0 when the rays are parallel -> high uncertainty
•   if (Math.abs(denom) < 1e-6) {
•     return { point: null, gap: Infinity }; // near-parallel: report as unreliable, don't
divide
•   }
•   const daR = da.dot(r);
•   const dbR = db.dot(r);
•   const s = (daDb * dbR - daR) / denom;    // param along ray a
•   const t = (dbR - daDb * daR) / denom;    // param along ray b
•   const pa = a0.clone().addScaledVector(da, s);
•   const pb = b0.clone().addScaledVector(db, t);
•   const point = pa.clone().add(pb).multiplyScalar(0.5);
•   const gap = pa.distanceTo(pb);           // shortest distance between the two rays
•   return { point, gap };
• }

```

For **N rays**, solve the least-squares "point minimizing the summed squared perpendicular distance to all rays" directly (build and solve the 3×3 normal equations from $\mathbb{I} = d \cdot d^T$ per ray), fold each depth point in as an extra weighted observation, and derive your **uncertainty metric** from the residual spread — exactly the metric you designed in Phase 1, now also driving the baseline coaching.